Software - Architecture, Design, Test, and...

Troels Mortensen

May 11, 2026

Contents

Todo list

Part I

Introduction

Chapter 1

Case Study Domain Overview

Throughout this book, we use a single running case study called *Abyssal Watch*. The setting is a near-future world where deep-ocean Breaches to other dimensionsare monitored continuously because they can produce large biological anomalies, known as Kaiju (yes, clearly inspired by Pacific Rim, and probably other lore I am not familiar with).

The operational challenge is not only to detect anomalies, but to coordinate people, vessels, and sensing infrastructure under uncertain conditions and intermittent connectivity.

The *Abyssal Watch* initiative operates through Command Carriers that deploy Scout Vessels, collect Observations, and oversee Missions in assigned sectors. Some operations are routine surveys, while others become high-pressure tracking missions when a Kaiju signature emerges from a Breach. This gives us a domain with plenty of possibilies and I can create all kinds of interesting examples and scenarios. As a result, it is a useful foundation for discussing boundaries, interactions, trade-offs, and design decisions across multiple architectural styles.

In practical terms, we will revisit the same domain from different viewpoints: domain modeling, decomposition, data flow, integration, testing strategy, and various design strategies. Keeping one consistent context helps us focus on architectural reasoning rather than re-learning a new problem in each chapter.

1.1 Domain at a Glance

The core domain entities are intentionally small and stable:

- **Command Carrier**: mobile operations base that coordinates deployments and sector monitoring.
- **Scout Vessel**: deployed platform used to survey Breaches and track Kaiju signatures.
- **Crew Member**: personnel assigned to manned Scout Vessels.
- **Breach**: deep-ocean origin point monitored for abnormal activity.
- **Kaiju**: tracked biological anomaly associated with Breach activity.
- **Detection Buoy**: stationary sensing node contributing distributed observations.
- **Mission**: operational lifecycle from planning through deployment and closure.
- **Observation**: recorded sensor signal used for detection, confirmation, and tracking.

These concepts are sufficient for an introductory model and are intentionally kept free of implementation details. We can enrich behavior later where a chapter needs deeper constraints, state transitions, or data structures.

1.1.1 High-Level Domain Relationships

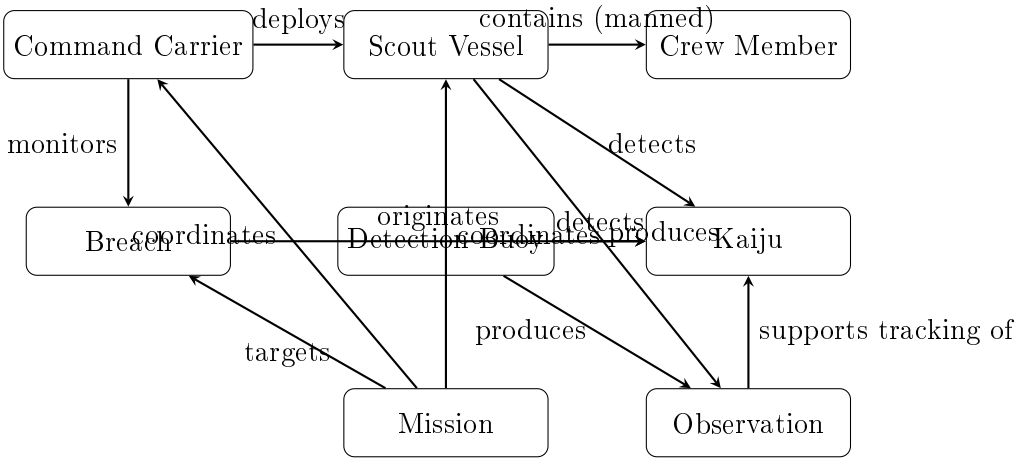


Figure 1.1: High-level domain model for the Abyssal Watch case study.

The figure in Figure 1.1 is deliberately conceptual: it captures the principal entities and operational relationships, but omits attributes and storage concerns. This level of detail is enough to establish a shared mental model for the rest of the book.

fix the diagram, what even is this

Chapter 2

Hexagonal Architecture

Hexagonal Architecture, also known as *Ports and Adapters*, was introduced by Alistair Cockburn to protect application behavior from infrastructure concerns.[1] The key idea is simple: we keep application logic in the center, and let external technologies connect through explicit boundaries. These boundaries are called *ports*, and are generally just interfaces in your code. A modern introduction with practical examples is provided by Huttunen, and has inspired parts of this chapter.[4]

In short, the point of hexagonal architecture is to keep a primary focus on the business logic, and allow external technologies to "plug in" to the core system, through interfaces. It is a "solve the business problem"-first approach, rather than a "technology"-first approach.

Whether the system is activated through a GUI, a Web API, a CLI application, or something else, is considered a technical "detail" of the system.

Whether data is stored in a database, a file, or something else, is considered a technical "detail" of the system.

2.1 Why We Use Hexagonal Architecture

Many systems start in a layered style, but business rules often leak into controllers, persistence code, ORMs, and other technological details. Basically just places where it does not belong. This coupling makes change expensive: replacing persistence technology, adding a new interface, or testing behavior can require touching several technical layers, even if we just want to verify the core business logic is working as expected.

Similarly to a 3-layered architecture, we want to separate the concerns of the application into different parts. We just approach it slightly differently.

Hexagonal architecture addresses this by separating concerns into an *inside* and an *outside*. The inside contains the behavior we care about, while the outside contains delivery and infrastructure details. Comparing to the 3-layered architecture, the presentation and persistence will be considered the "outside" of the application. We start with the core hexagon, see Figure 2.1.

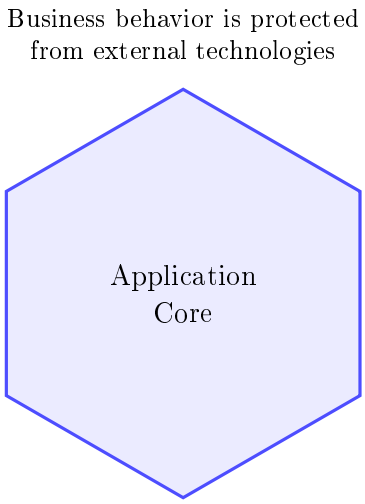


Figure 2.1: The first step is to define a protected application core.

2.1.1 Purpose of the Inner Hexagon

The hexagon is a visual metaphor, not a geometric requirement (which probably would not make sense). Cockburn has said it was simply a shape that was not yet associated with anything in software architecture. It reminds us that the core should be reachable from many directions, or input devices, without being rewritten for each one. Whether requests come from a GUI, a Web API, a CLI application, a batch job, or an automated test, they should trigger the same use-case behavior.

2.2 Step 1: Start with Application Logic

At the beginning, we model only application behavior, for example in `ConfirmKaijuDetectionService` or `TrackKaijuUseCase`. These classes are responsible for business logic, i.e. executing a use case and enforcing its rules. Commonly such a class is suffixed "Service", or "UseCase", or "Handler".

At this stage we do not need to commit to `Spring`, `ASP.NET`, `PostgreSQL`, or a message broker. We intentionally start here because architecture decisions become clearer when behavior is visible first.

For the rest of the chapter, we use one running Java example: `Confirm First Kaiju Detection`. This gives us one consistent slice from driving adapter to core behavior to driven adapters.

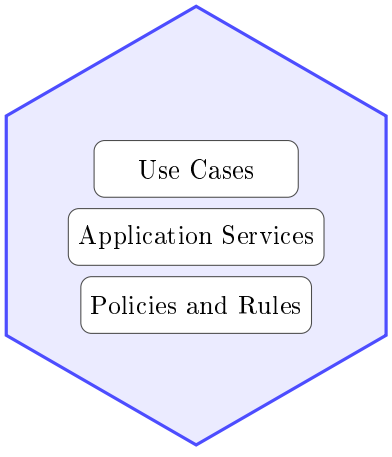


Figure 2.2: Initial core structure focused on use cases and policies.

The content of the application core has several names. Often, classes in here are called "application services" or "use cases". And are therefore suffixes with "Service" or "UseCase". The responsibility of these classes is to execute a use case and enforce its rules. For example, `ConfirmKaijuDetectionService` is a service that confirms a Kaiju detection. The class will then expose a method for this particular use case, and implement the logic for it.

Logic include many things:

- Validating input data,
- Applying business rules,
- Coordinating across output ports,
- and persisting the results.

As such, the application core is the heart of the application, and over time, it will grow to contain the majority of the application's logic, placed in these service classes.

2.2.1 What We Delay on Purpose

We deliberately delay technology choices. This gives us two benefits:

- We can validate business behavior early without waiting for full infrastructure setup.
- We avoid coupling use-case logic to framework types and transport protocols.

2.2.2 Java Skeleton of the Core

The initial core can be expressed as a use-case-focused Java API and service implementation.

First we need the "entry" interface, i.e. the port that will be used by the driving adapters to invoke the use case. It declares a piece of behaviour the core can execute.

```
1 public interface ConfirmKaijuDetectionPort {
2
3     KaijuId confirmDetection(
4             ConfirmKaijuDetectionCommand command
5         );
6
7 }
```

Next, we have the implementation of the port. It is a service class that implements the interface and contains the use-case logic. Here, we confirm a detection by linking a new `Kaiju` to its origin `Breach` and persisting it through `SaveKaijuPort`.

There is also a second output port, `PublishKaijuConfirmedPort`, implemented by a driven adapter that publishes a mission event. It is similar to the Observer Design Pattern. This event can trigger downstream actions such as command alerts or tracking workflows.

IN-REF: add reference to Observer Pattern

```java
public final class ConfirmKaijuDetectionService implements
    ConfirmKaijuDetectionPort
{
    private final LoadBreachPort loadBreachPort;
    private final SaveKaijuPort saveKaijuPort;
    private final PublishKaijuConfirmedPort
        publishKaijuConfirmedPort;

    // Constructor omitted for brevity.

    @Override
    public KaijuId confirmDetection(
            ConfirmKaijuDetectionCommand command
        )
    {
        Optional<Breach> breach = loadBreachPort
                .loadById(command.originBreachId())
                .orThrow();

        Kaiju kaiju = Kaiju.confirmed(command.codename(), command
            .classification(), breach.id());
        saveKaijuPort.save(kaiju);
        publishKaijuConfirmedPort.publishConfirmed(kaiju.id(),
            breach.id());
        return kaiju.id();
    }
}
```

2.3 Step 2: Add Driving Adapters and Input Ports

Once the core behavior exists, we connect ways to invoke it. These are *driving adapters* (also called primary adapters). Their responsibility is to translate external input into calls on input ports, which are interfaces representing use cases.

Obviously, multiple driving adapters can be used to invoke the same use case. For example, we could have a REST controller, a CLI handler, and a GUI controller all invoking the same use case. This is why we need the input port, to abstract the use case from the driving adapter. In the below diagram I have kept it simple and reduced the number of arrows from driving adapters to input ports.

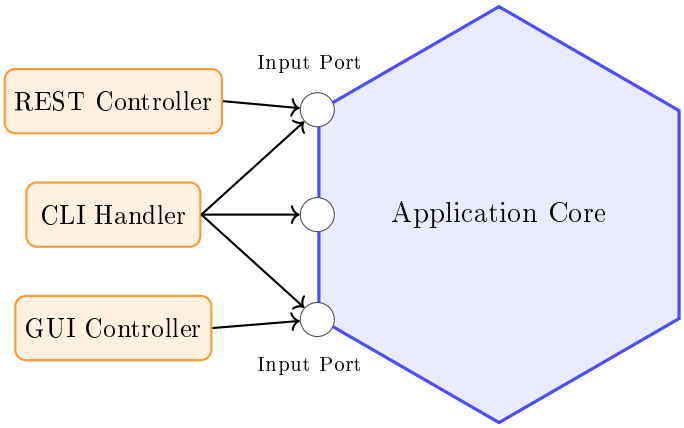


Figure 2.3: Driving adapters call input ports to invoke application behavior.

We can simplify the diagram a bit, by just adding a new hexagonal layer to the left side to represent the driving adapters:

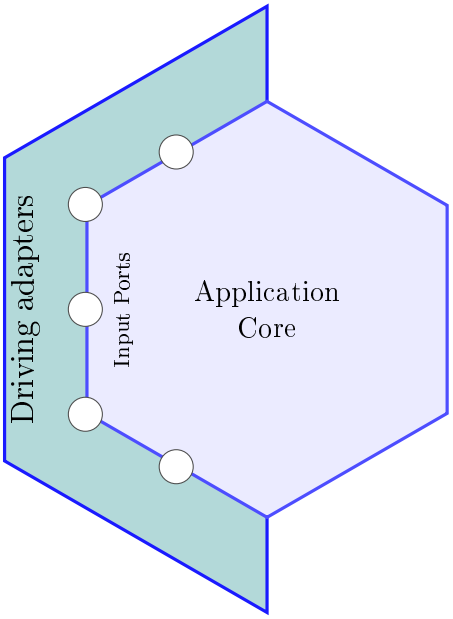


Figure 2.4: Driving adapters to the left of the application core.

2.3.1 Examples of Driving Adapters

Common driving adapters include:

- a REST controller translating HTTP requests into use-case calls,
- a CLI handler translating command arguments into use-case calls,
- a GUI presenter/controller translating user events into use-case calls,
- a message consumer translating broker events into use-case calls,
- and automated tests that call input ports directly.

2.3.2 Input Port and Driving Adapter in Java

The driving adapter talks only to the input port and maps transport-specific models to use-case models:

```java
@RestController
@RequestMapping("/kaiju-detections")
public class KaijuController {
    // Dependency injected.
    private final ConfirmKaijuDetectionPort
        confirmKaijuDetectionPort;

    @PostMapping
    public ResponseEntity<String> confirm(@RequestBody
        KaijuDetectionRequest request) {
         ConfirmKaijuDetectionCommand command =
                 new ConfirmKaijuDetectionCommand(
                         request.codename(),
                         request.classification(),
                         request.originBreachId());
         KaijuId kaijuId = confirmKaijuDetectionPort.
            confirmDetection(command);
         return ResponseEntity.accepted().body(kaijuId.value());
    }
}
```

2.4 Step 3: Add Driven Adapters and Output Ports

The core eventually needs support from external capabilities such as storage, notifications, or third-party APIs. We model these needs as output ports, and implement those ports with *driven adapters* (secondary adapters). This is where dependency inversion becomes concrete: the core depends on interfaces (and *owns* those interfaces), while infrastructure depends on those same interfaces.[3]

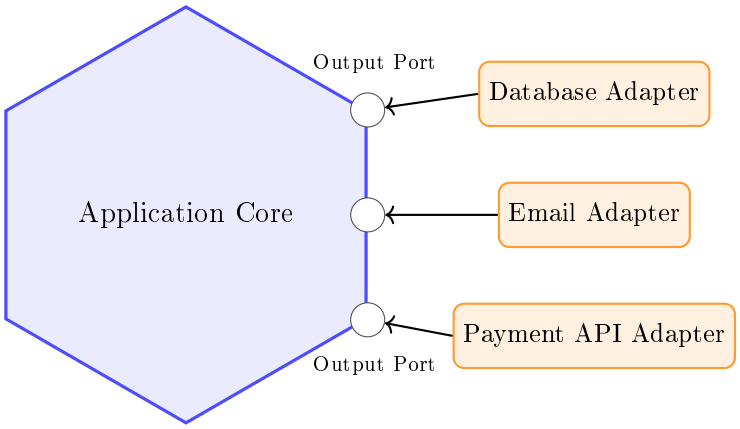


Figure 2.5: Driven adapters implement output ports for infrastructure concerns.

Again, we can simplify the diagram a bit, by just adding a new hexagonal layer to the right side to represent the driven adapters:

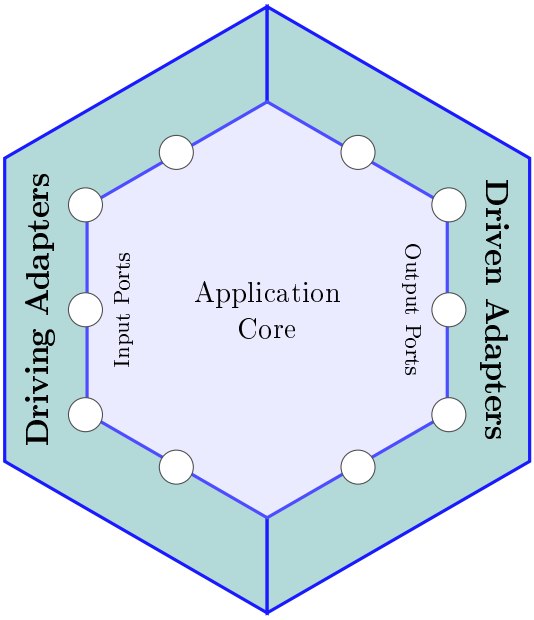


Figure 2.6: Driven adapters to the right of the application core.

And, again, we can update the diagram to include a thin hexagon around the core to represent the ports of the core. Just to clarify that there is a "border" between the core and the adapters, on both sides.

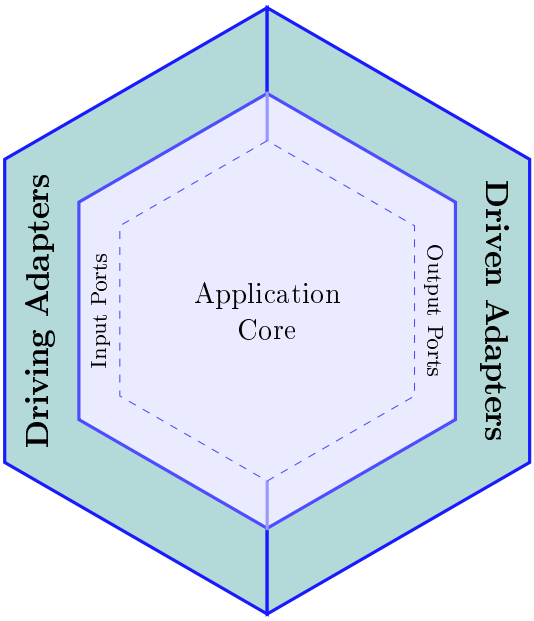


Figure 2.7: Both sides of the hexagon, with the core in the middle.

Notice how the inner hex is dashed, to indicate a "border" between the core and its ports, but to still indicate that the ports are *inside* the core.

2.4.1 Examples of Driven Adapters

Typical driven adapters are:

- a repository adapter for `PostgreSQL` or `MongoDB`,
- an email adapter for `SMTP`,

- a payment adapter for an external payment service API, like Stripe or PayPal,
- and a publisher adapter for a message broker, like Kafka or RabbitMQ.

2.4.2 Output Ports and a Driven Adapter in Java

On the driven side, the core defines output ports, and infrastructure implements them.

Let's start with the first output port, `LoadBreachPort`. It is used to load a Breach from storage. I use `Optional` to indicate that the Breach may not be found.

IN-REF: add reference, when Optional pattern is written

```
public interface LoadBreachPort {
    Optional<Breach> loadById(BreachId breachId);
}
```

Then the next port to save a Kaiju entity to the storage (database or other storage). I keep the interfaces small here. But sometimes you may want to group multiple related operations into a single interface. For example, you could have an interface which contains both the save and load operations.

```
public interface SaveKaijuPort {
    void save(Kaiju kaiju);
}
```

And an implementation of the latter port. Here we use `Spring Data JPA` to save the Kaiju to storage. If you are unfamiliar with `Spring Data JPA`, you can think of it as a library that provides a way to interact with a database, without explicitly writing SQL queries.

```
public final class JpaKaijuRepositoryAdapter implements
    SaveKaijuPort {
     // Dependency injected.
     private final SpringDataKaijuRepository repository;

     @Override
     public void save(Kaiju kaiju) {
         repository.save(KaijuEntity.fromDomain(kaiju));
     }
}
```

The purpose of this adapter is to hide the database implementation details from the core.

2.4.3 Mapping Boundaries in Java

Mapping code belongs in adapters, because adapters translate between outside models and core models. For simplicity, I use here record classes to represent the commands and requests.

```
public record KaijuDetectionRequest(String codename,
                                    KaijuClassification
                                       classification,
                                    BreachId originBreachId) {}

public record ConfirmKaijuDetectionCommand(String codename,
                                           KaijuClassification
                                              classification,
                                           BreachId
                                              originBreachId) {}

ConfirmKaijuDetectionCommand command =
        new ConfirmKaijuDetectionCommand(
                request.codename(),
                request.classification(),
                request.originBreachId());
```

```
@Entity
public class KaijuEntity {
    @Id
    private String id;
    private String codename;

    public static KaijuEntity fromDomain(Kaiju kaiju) {
        KaijuEntity entity = new KaijuEntity();
        entity.id = kaiju.id().value();
        entity.codename = kaiju.codename();
        return entity;
    }
}
```

2.4.4 Dependency Rules

We must remember the dependency rules of this architecture

- The core should not depend on the adapters.
- The core defines the ports (interfaces) that the adapters must implement.
- The adapters should depend on the core.
- The adapters should not depend on each other.

This means that dependencies should only go from the outside towards to core. When we then organize the code into packages or modules, we apply these dependency rules by being careful about which packages a class can import.

DIAGRAM: add diagram showing the direction of dependencies. Inner hex and outer hex divided vertically into two parts, with the core in the middle.

With this layout, we keep dependency directions explicit:

- `domain` and `application` do not depend on `adapters`.
- `adapters.in.*` depend on input ports in `application.port.in` (i.e. the interfaces providing access to the core). Classes in this package are our driving adapters.
- `application` depends on output ports in `application.port.out` (i.e. the interfaces that the core defines, to be implemented by the adapters).

- `adapters.out.*` implement output ports and may depend on framework libraries. Classes in this package are our driven adapters.
- `config` performs wiring and can see both core and adapters.

2.4.5 Requires vs Provides Contracts

The most important design choice on the driven side is contract ownership. This is where the hexagonal architecture differs from layered architecture. In hexagonal architecture, the core defines contracts by what the use cases *require*. For example, a core service may depend on `LoadBreachPort` and `SaveKaijuPort`, because those capabilities are needed to complete the use case and preserve the Kaiju-origin invariant.

In many layered implementations, interfaces are shaped by what the persistence layer *provides*, for example a repository abstraction aligned to ORM operations. That approach can still be effective, and we can still mock those interfaces. The difference is who drives the abstraction: in hexagonal architecture, required core behavior drives the port definition; in provider-oriented layering, available infrastructure capabilities often drive the interface definition.

This distinction matters during development. With core-owned output ports, we can implement and test use cases before deciding the final database, broker, or external API integration.

DIAGRAM: I don't like this diagram

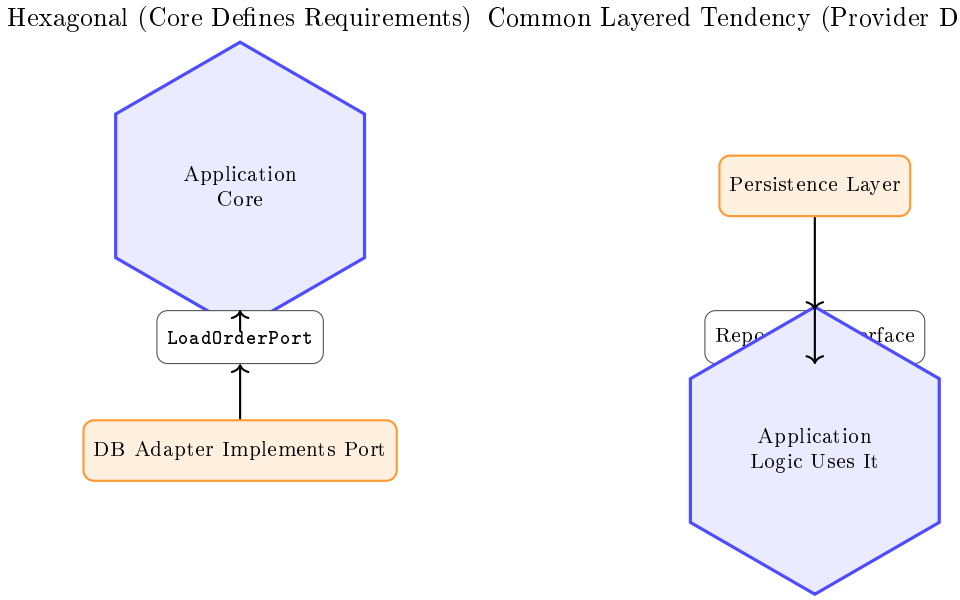


Figure 2.8: Contract ownership comparison: core-defined required ports vs provider-defined interfaces.

2.5 Step 4: Evolving the Core with a Domain Model

In some projects, the first hexagonal version uses lightweight application services and simple data structures. That is a valid starting point. As rules become richer, we often add an explicit Domain Model to keep invariants and behavior close to business concepts.

Cockburn's original model did not include an explicit domain model, but if you were to study this architecture online, you will find that it is usually included.

2.5.1 Why the Domain Model May Come Later

In the original hexagonal architecture, there was no domain model. You *can* write software without a domain model of classes and objects. You might go with a simple *data model*, meaning

a collection of classes that represent the data you are working with. I realize some people *do* consider this a domain model, but it is an *anemic domain model*, meaning only data but no behavior. You can also go even more primitive and just use simple data structures, but I really don't think anyone wants that.

About the same time as hexagonal architecture was introduced, the Domain-Driven Design (DDD) paradigm was introduced by Eric Evans [2]. And so, shortly after, the original hexagonal architecture diagram was extended with a domain model.

Adding a domain model later is not a failure of architecture; it is often a sign of responsible evolution. We first establish boundaries and interaction patterns, then deepen the model when complexity justifies it.

We can illustrate the addition of the Domain Model by introducing a new hexagon, inside the core. This now means that the Service classes, in Application Core, can act upon the Domain Model, and that the Domain Model has no dependencies on any outer hexagon.

Figure 2.9 shows the updated diagram:

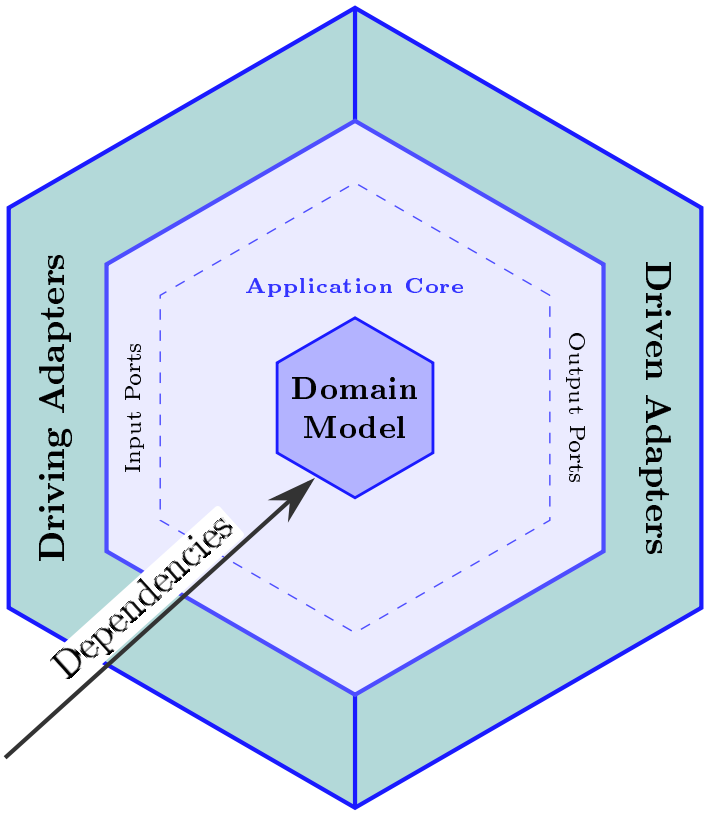


Figure 2.9: The domain model inside the core.

For the sake of clarity, I have added a nice arrow to indicate that the dependencies point inward. Outer hexes (layers) may depend on inner hexes, e.g. by importing and using classes or functions from the inner hex (package/module).

Another way to diagram the final architecture is the more common way of using boxes for each layer/part. This is shown in Figure 2.10:

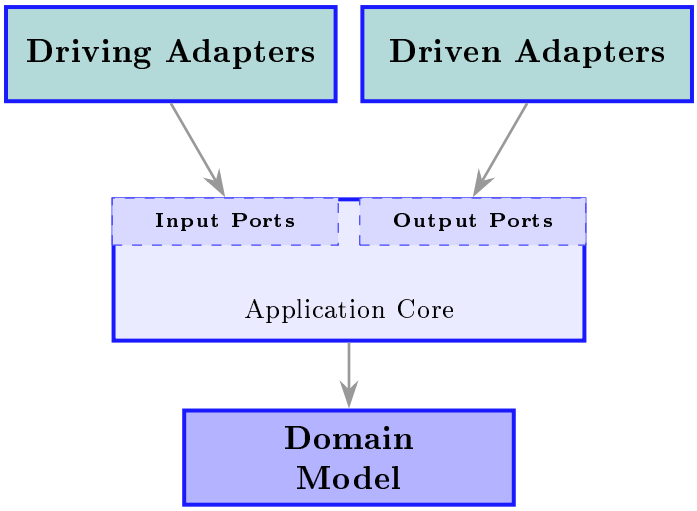


Figure 2.10: The final architecture as boxes.

2.6 Package Layout

An important aspect of *architecture* is the organization of the code into packages and modules, along with the dependencies between them. Most architectural styles will have a recommended superficial package layout, so let us explore how we can organize an application using hexagonal architecture.

A package convention gives us a practical guardrail. We can often organize our code by architectural role, so that core code remains stable while adapters can change. There are many ways to do this, I will give an overall recommendation, and then two suggestions on how to organize the aplication hex: layers and feature-folders.

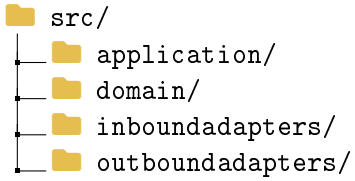
2.6.1 Overall structure

We generally have three parts to a hexagonal architecture: the core, the driving (inbound) adapters, and the driven (outbound) adapters. So, that's where we start: create three packages (or modules).

```
src/
├── application/
├── inboundadapters/
└── outboundadapters/
```

Many applications, across many architectural styles, will actually have a similar package structure, perhaps just differently named. Sometimes the application is called "core", sometimes "services". The same responsibility has been named different things over the years.

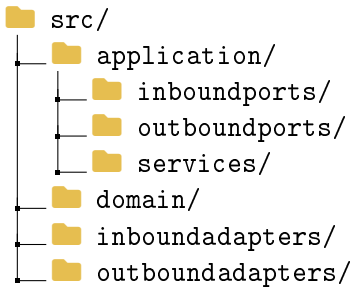
Next, we also need the domain. We can place it inside the application package, to represent that the domain hex is inside the application hex. Or, we can put it in a separate package. Below, I will show the latter:



Then, we have to further organize the application hex, or package. As mentioned earlier, I will show two approaches: layers and feature-folders.

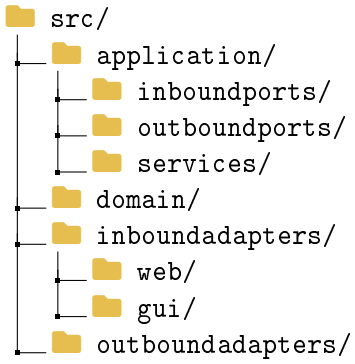
2.6.2 Application Layers

Let's start with layers. The application hex consists of service classes (the classes performing the use cases), inbound ports (the interfaces exposing the use cases to the outside world), and outbound ports (the interfaces defining the capabilities the core needs from the outside world). If we go with the layered approach, we group files by their type of responsibility, or category. That means three sub-packages:

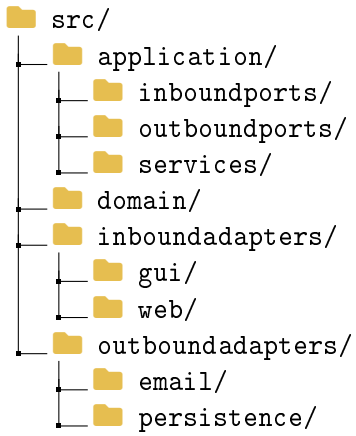


This is the basics. Your application may need other kinds of classes, which do not fit into one of the three categories.

I will not go into too precise details about the adapters here. In your `inboundadapters` package, you will have classes that lets the outside world interact with the application. This could be a Web API controller, a CLI handler, a GUI controller, a message consumer, or something else. Let's just say that our application exposes a REST API, and also has a GUI. I will make two sub-packages in the `inboundadapters` package: `web` and `gui`.

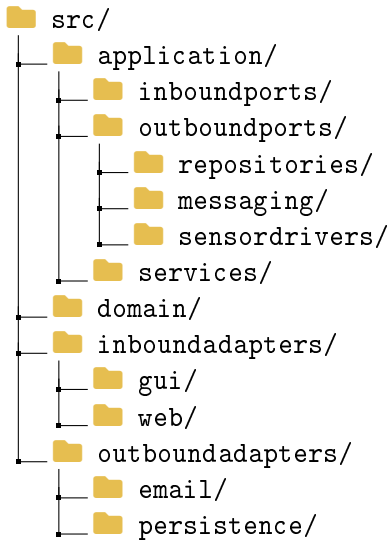


What about outbound adapters? Again, we have many categories, try to organize your code by your needs. For example, you may have packages for `persistence`, `events`, `notifications`, `messaging`, `email`, `logging`, `sensordrivers`, `externalAPIs`, etc. Let's just add a sub-package for `persistence` and `email`:



Further subdivision of your application is of course possible, and as the capabilities of the application grow, you will likely need to add more sub-packages. Here, you will have to decide what is the best way to organize your code.

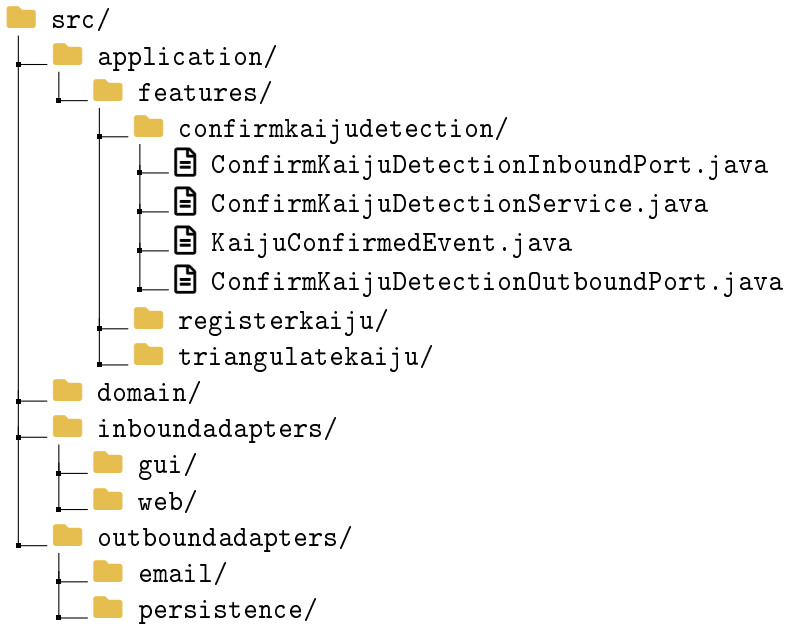
For example, you may want a group of `repositories` ports, which are used to perform CRUD operations of entities on the database. You make a package for that: `outboundports/repositories`. Or you have ports for messaging, or sensor drivers. So, we can update the package structure to:



2.6.3 Feature Folders

Another way to organize the application is to use feature folders. In the previous approach, a service class in the `services` package will expose an inboundport, located in the `inboundports` package. And it may depend on a number of outbound ports, located in the `outboundports` package. That means, classes which work closely together to implement a feature, are spread across different packages, and navigating between those files may become cumbersome. That, at least, is the concern, which leads to the feature folder approach. So, instead, we organize the code by feature, or use case, a package per. And the service class, with its inbound and outbound ports, will be located in the feature package.

Here is an example, I have just updated the `application` package to use feature folders:



Notice how each feature package clearly describes the feature through its name. It then contains whatever classes and interfaces are needed to implement the feature. The adapters, of course, are still placed outside. Just pretend the other feature packages also contains relevant files.

You might similarly organize your adapter layers by feature, so in the `outboundadapters` package, you will have sub-packages `confirmkaijudetection`, `registerkaiju`, and `triangulatekaiju`. And the same for the `inboundadapters` package.

2.6.4 Layers vs Feature Folders

So, which approach should you choose? Both are valid, and both have their own advantages and disadvantages.

Layer folders: pros

- clear architectural map for newcomers, because `services`, `inboundports`, and `outboundports` make dependency direction explicit,
- easy to enforce package-level architecture rules and detect layering violations early, by simply looking at import statements,
- consistent structure across many features, which improves predictability in larger teams,
- centralized role-based code can simplify governance and cross-cutting standards,
- architecture tests are often straightforward, because package naming aligns with technical roles.
- reuse of outbound ports across services is easier, because multiple service classes in the service package can depend on the same outbound port.

Layer folders: cons

- feature code becomes scattered across several packages, increasing navigation overhead,
- implementing/updating one use case often requires changing files in multiple locations,

- classes that evolve together can drift apart because they are separated by technical category,
- extracting one capability later can be harder when its parts are distributed,
- you risk tying services together if they depend on the same outbound ports. One services requires a change to the port, now the other service may encounter issues.

Feature folders: pros

- high feature cohesion: service and ports for one use case live in one place,
- faster development flow, because most edits for a feature are local to one folder, or even just one file,
- ownership maps naturally to business capabilities rather than technical layers,
- clear organization by feature, making it easier to understand what the application does,
- updating a feature requires changes in *one* package,
- pull requests are often easier to review, because changes stay focused on one capability,
- feature-oriented boundaries can make future extraction or modularization easier, if you want to move towards a `modular monolith`, `vertical slice architecture`, or `microservices`.

IN-REF: add reference to modular monolith, vertical slice architecture, and microservices

Feature folders: cons

- repeated technical patterns can appear across features (validation, mapping, error handling) violating the DRY principle,
- conventions may drift if teams do not actively maintain shared standards, each developer may invent their own conventions,
- shared abstractions/ports need deliberate placement to avoid duplication or fragmentation, e.g. repository ports may be shared across features, and is often placed in a `common` package, which can grow larger and larger,
- global refactoring of cross-cutting concerns may touch many feature folders, which can be difficult to manage,
- you may consider organizing adapters similarly, which may make you wonder `why split a feature folder into three feature folders in the first place`? And that's a valid question. Or if you still organize your adapters by technical role, you are mixing two approaches. Will that be confusing?

A practical choice guideline

- prefer **layer folders** when architectural governance, role consistency, and shared technical abstractions are the main concern,
- prefer **feature folders** when business workflows change frequently and teams deliver vertical slices,
- and consider a **hybrid**: keep top-level feature folders, while applying light internal layering inside each feature where useful.

2.7 Example

I hope by now you have some kind of understanding of the architecture. Comparing it to the layered architecture, the main difference is primarily about where the outbound ports (interfaces) are placed. The rest is pretty similar to the layered architecture. So, I think it is time for another example. It will perhaps be a bit elaborate, because I had fun developing the use case. It is about using sensors to triangulate the position of a suspected Kaiju.

I will describe the use case, introduce the classes and ports involved, their relationships, and the package layout. I will also provide relevant code snippets. The focus will be on the structure of the code, primarily around the application hex, i.e. the service class. The inbound adapter could just be a Web API endpoint, which is not particularly interesting. And some of the outbound adapters would become too technical to be interesting, for example how to poll the sensors for data.

Furthermore, the specific purpose of the hexagonal architecture is to focus on the application logic, and treat the external adapters as technical details. You should notice that the shape of the outbound port is defined by what the application service needs. An adapter will then have to figure out how to actually achieve that, for example by connecting to a sensor and polling it for data.

2.7.1 Use Case: Triangulate Kaiju Position

Here is the use case: Suppose we suspect a Kaiju has exited a Breach, and we want to triangulate its position. We have sensors placed across the sea, sensors of various types: Seismic, Acoustic, Radiation, Thermal, etc. We want to select a relevant number of sensors in the suspected area, and use their readings to triangulate the position of the Kaiju. The result is an estimated area of the Kaiju, and we can then launch a Scout Vessel to investigate, and possibly visually confirm the Kaiju.

2.7.2 Visual explanation

As mentioned earlier, I picked this case because it is a bit elaborate, and I had fun developing it. It also (I hope) illustrates the concept of the hexagonal architecture well. But it does require me to introduce a bit of domain terminology first, so let's start with that.

Let's assume that any sensor, regardless of type, can return a reading which includes a direction (relative to the sensor's location), and a distance. This would result in a point on a 2d plane, i.e. the position of the Kaiju. However, sensors have some uncertainty in their readings, for example because of heavy waves, which may distort readings, or sea floor conditions like reefs, rocks, sand, etc, which may also distort readings. Therefore, we do not get a point, but an area. This area is called a *Annular Sector*.

And what exactly is an annular sector? Observe Figure 2.11.

A) Here we have a sensor, in the bottom left corner.

B) Here is the Kaiju, we suspect has appeared.

C) The sensor will produce a measurement, i.e. a vector, relative to the sensor's location.

D) The sensor has angular uncertainty, of some degree θ.

E) The direction is therefore a circular sector.

F) The sensor has distance uncertainty, of some radius `r`.

G) This results in two circular sectors (min and max distance).

H) In the end, we end up with the annular sector (red shaded area).

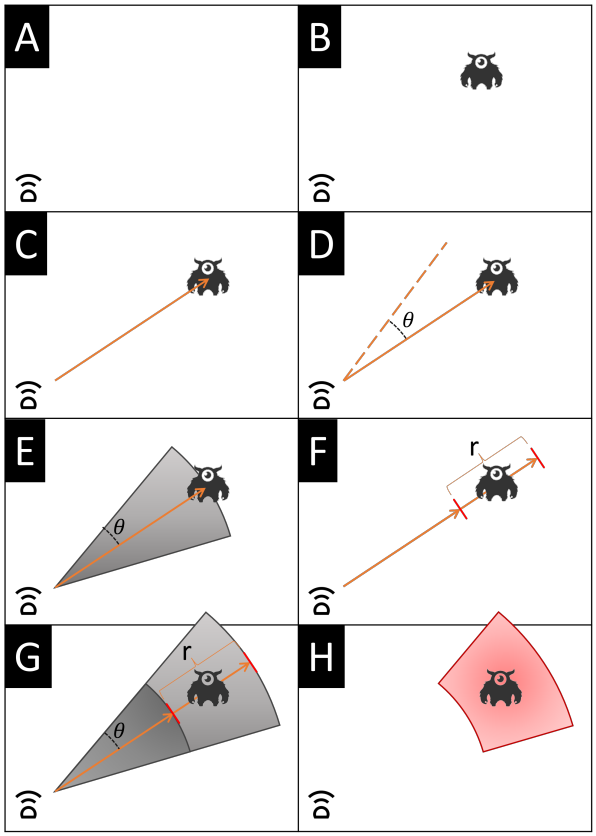


Figure 2.11: Deriving the annular sector.

Now, let's assume we have multiple sensors, each producing an annular sector. We can then use the intersection of these sectors to get a more accurate estimate of the Kaiju's position. This is the principle of triangulation. See Figure 2.12. Based on the sensor data from 5 different sensors, we have narrowed down the possible position of the Kaiju to a small area, marked in purple.

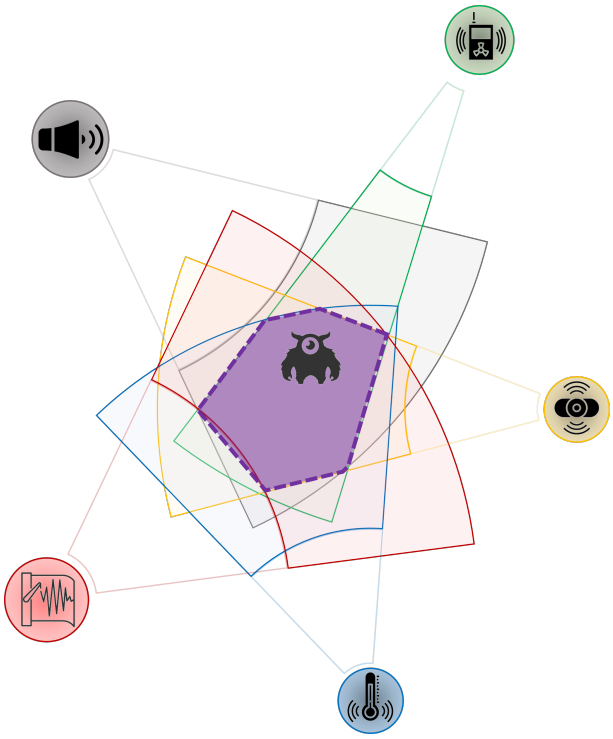


Figure 2.12: Triangulation of multiple annular sectors.

Good, I trust that the idea of the use case is now clear. The input to the use case is a suspected location: a point, plus a radius. All sensors within that radius will be polled. The output is an estimated area, wherein the Kaiju is likely to be. Another use case could then be to visually confirm the Kaiju, by launching a Scout Vessel to the estimated area. But, one thing at a time. Let's start with the triangulation use case.

I will try to balance the complexity by abstracting away some technical details, like how to calculate the output area from the array of annular sectors. Let's just assume, we can write a method that takes an array of annular sectors, and returns an estimated area, wherein the Kaiju is likely to be.

2.7.3 Classes involved

Let's look at the classes involved, I will provide a brief overview. Follow up sections will show the structure or location of the code, in relation to the hexagonal architecture. As this architecture focuses on business logic first, let's derive the classes from the use case by moving from inner hexes to outer hexes.

Domain

Let's start with the inner most hexagon, the domain.

1. I need a domain entity to represent the `Sensor`. It will contain various values, and some of these will be represented by other classes. You will see in the code later.
2. To avoid `primitive obsession`, I am going to model this with a few other classes, like `Location`, `Longitude`, `Latitude`, `SensorId`, `SensorType`, and perhaps more.

Application

Then we move on to the application hexagon, where I define services and ports.

1. I need the `Service` class to implement the use case, `TriangulateKaijuPositionService`. It will contain the business logic, and will depend on the `Sensor` entity. It will have a method for executing the use case, which takes a `Location`, and a `Radius`. It will return a class, `EstimatedArea`, to represent the estimated area of the Kaiju. This `EstimatedArea` is not a domain entity, but just a simple object with a few properties.

2. My `Service` class will need to be able to retrieve all sensors within a given radius. I will create a `SensorRepository` port for this. Remember, a `Repository` is generally a port which can perform CRUD operations on an entity against a data storage (e.g. database), and the `Read` operation may optionally take filter parameters, like location and radius. Alternatively, I would define a dedicated port just for this use case.

3. My `Service` class will need to be able to poll the real sensors for data. The `Sensor` entity is just a *representation* of a real sensor somewhere out in the ocean. The domain is isolated from the outside world, and therefore nothing in the domain can access the outside world, or more specifically, my `Sensor` entity cannot access the real world sensor. Instead, I need a port to represent the ability to poll the real sensor for data. I will call this port `SensorDriver`. There will be multiple adapters, which can connect to the real sensors.

4. Because I have different types of sensors, based on the `Sensor`'s `SensorType`, I there will be a specific adapter of the `SensorDriver` port for each sensor type. I need a way to figure out which adapter to use. Here comes an interesting problem. I have multiple adapters to the same port, the `SensorDriver` port. How do I know which adapter to use? I am going to introduce a `SensorDriverFactory` port, which will be responsible for creating the correct adapter based on the `SensorType`.

5. I also need to expose the use case through a port, so I create an interface for the service class, let's call it `TriangulateKaijuPositionUseCase`.

Inbound Adapters

We need access to the system, so we can execute the use case. I will create a Web API endpoint for this, but it could also be a CLI action, or a GUI button, or a message consumer, or something else. This is a technical detail, and therefore placed outside the core. And also less interesting to write about. So, let's just say I have a Web API endpoint, which will be responsible for invoking the use case.

Outbound Adapters

These adapters provide access to the outside world. Either so my service class can send data out of the system, or pull data into the system.

1. I need an adapter for the `SensorRepository` port. Let's assume several use cases need to execute CRUD operations on sensor entities, so there is already an adapter for this port.

2. I need an adapter for the `SensorDriver` port. Actually, as I mentioned earlier, I need multiple adapters for this port, one for each sensor type. Different sensor types may require different protocols to communicate with the real sensor.

3. I need an adapter for the `SensorDriverFactory` port. It will have a method for creating the correct adapter based on an input of a `SensorType`. It then figures out which adapter to create, and return it. Remember, all my SensorDriverAdapters will implement the `SensorDriver` port, so they will all have the same interface.

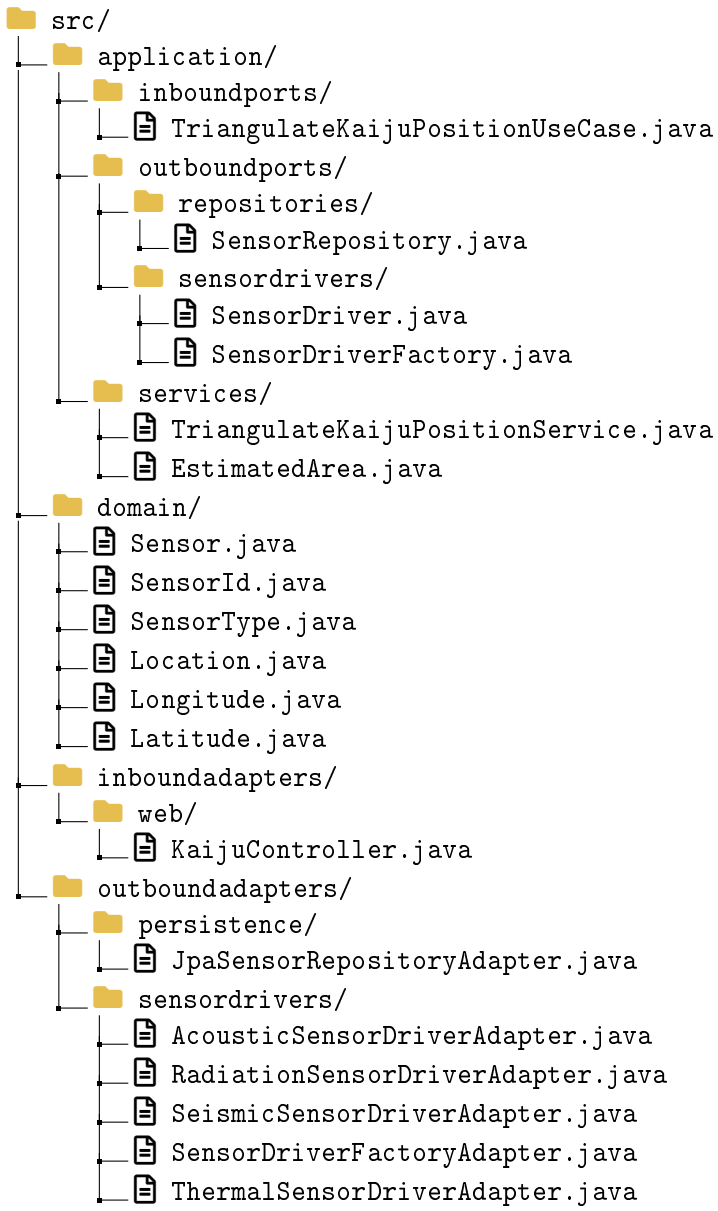
I just need to make sure it is clear, why the `SensorDriverFactory` adapter is needed, and why it is located here in the outbound adapters. My `Sensor` entity just knows its type, but the specifics about how to connect to the real sensor is not part of the domain, we use the `SensorDriverAdapters` to do that. And these are located in the outbound adapters.

My Factory needs to know about the concrete adapters, and because the Application hex cannot know about the Outbound hex, the Factory *must* be placed in the Outbound hex, where it can reference, and thereby create, the concrete adapters. My Application hex still needs access to this adapter, so we define the port in the Application hex.

These are the classes involved, so let's now look at the package layout.

2.7.4 Package layout

For this example I use the layered organization of the application hex, that is, separate sub-packages for `inboundports`, `outboundports`, and `services`. The outbound ports are further grouped by category, so that repository ports and sensor driver ports each live in their own sub-package. Domain types and adapters follow the same overall structure introduced earlier in this chapter.



Notice how the `SensorDriverFactoryAdapter` sits next to the concrete `SensorDriver` adapters in `outboundadapters/sensordrivers/`, while its port `SensorDriverFactory` stays in `application/outboundports/sensordrivers/`. The factory must be able to reference each concrete adapter to instantiate the right one, which is only allowed on the outbound side. The next subsection shows how these classes relate to each other.

2.7.5 Class diagram

2.7.6 Implementation

2.8 Testing Across the Hexagon Boundary

The separation of concerns gives us a practical testing strategy:[4]

- test application behavior through input ports without real infrastructure,
- test adapters against their technologies in focused integration tests,
- and keep end-to-end tests for wiring and critical user paths.

In short, we can focus on unit testing the behaviour of the core, by mocking the driven adapters.

IN-REF: add reference to mocking

2.8.1 Why Ports Improve Testability

Ports improve testability because they create explicit `seams` around behavior. An input port gives us a stable way to invoke a use case, while output ports let us replace infrastructure (outbound adapters) with in-memory test doubles . In practice, we execute a use case through an input port and provide fake implementations such as `LoadBreachPort` or `PublishKaijuConfirmedPort`. This keeps most tests fast and deterministic, because they verify business rules without requiring a running database, message broker, or HTTP communication.

IN-REF: add reference to seams

IN-REF: add reference to test doubles

This is not exclusive to hexagonal architecture. A disciplined layered architecture can also isolate dependencies behind interfaces and achieve good tests. The practical difference is that hexagonal architecture makes these seams a first-class design element around the core application, so isolated tests become the default rather than an optional refactoring.

2.8.2 Testing Slices with Java

In Java, we can keep most behavior tests focused on the core. Here is an example of testing a use case defined in a service class. We use a fake implementation of the repository and the event publisher. Both fakes implement the the necessary interface.

```
@Test
void confirmsDetectionAndPublishesEvent() {
    FakeLoadBreachPort loadPort = new FakeLoadBreachPort();
    FakeSaveKaijuPort savePort = new FakeSaveKaijuPort();
    FakePublishKaijuConfirmedPort publishPort =
                        new FakePublishKaijuConfirmedPort();

    ConfirmKaijuDetectionService service =
                    new ConfirmKaijuDetectionService(
                        loadPort,
                        savePort,
                        publishPort
                    );

    ConfirmKaijuDetectionCommand command =
                    new ConfirmKaijuDetectionCommand(
                        "Leatherback",
                        KaijuClassification.CATEGORY_III,
                        new BreachId("B-01")
                    );

    KaijuId id = service.confirmDetection(command);

    assertTrue(savePort.savedKaijuIds().contains(id));

    assertTrue(publishPort
                .publishedKaijuIds()
                .contains(command.originBreachId().value())
                );
}
```

2.9 Practical Trade-offs

Hexagonal architecture improves replaceability of the adapters and testability, but it introduces explicit interfaces and mapping code. For small CRUD-heavy systems, that can feel heavy. For systems with changing workflows, multiple interfaces, or expensive integration points, the added structure usually pays for itself over time.

2.9.1 Hexagonal and Layered: A Preview

Hexagonal and layered architecture are not enemies; both can be implemented well or poorly. A layered design with clear interfaces and dependency inversion can achieve strong testability. The practical difference we emphasize is orientation: hexagonal architecture centers on inside/outside boundaries and core-owned required ports, while layered architecture commonly centers on presentation, application, and persistence layers.

2.9.2 When Migration Is Worth It

Migration from layered to hexagonal is most useful when integration change is frequent, tests are dominated by slow end-to-end flows, or business rules are spread across controllers and repositories. If those symptoms are not present, a disciplined layered approach may remain a fine choice.

- If your focus is initially on the database, the layered approach makes sense.
- If your focus is initially on the business logic, the hexagonal approach makes sense, as you can focus on the services and the ports, and then add the adapters later.

In the end, the difference is not so much in the architecture, but in the focus.

2.9.3 Incremental Migration Steps

We can migrate incrementally instead of rewriting everything:

1. select one high-value use case (for example `ConfirmKaijuDetection`),
2. define an input port and required output ports for that use case,
3. move business logic into an application service behind the input port,
4. wrap current repository or API integrations as driven adapters,
5. add fast core tests that use fake output ports,
6. then repeat use case by use case.

What we essentially do here is create a port owned by the core, and then implement it with a driven adapter.

2.9.4 Common Implementation Mistakes

Even with a good structure, implementations can drift. Typical mistakes include:

- placing framework annotations in core domain models and use cases,
- exposing `JPA` entities directly across the core boundary,
- defining repository interfaces around provider capabilities rather than use-case needs,
- and embedding HTTP or SQL concerns in application services.

Bibliography

[1] Alistair Cockburn. *Hexagonal Architecture*. 2005. URL: `https://alistair.cockburn.us/hexagonal-architecture` (visited on 04/15/2026).

[2] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003. ISBN: 978-0321125217.

[3] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.

[4] Arho Huttunen. *Hexagonal Architecture Explained*. 2026. URL: `https://www.arhohuttunen.com/hexagonal-architecture/` (visited on 04/15/2026).